Taller de repaso y consolidación en C++: recursividad, búsqueda secuencial y punteros

Alexis Maier

E.E.S.T. N.º 1 «Eduardo Ader», Vicente López

Laboratorio de Programación (LPR) — 5.º año 3.ª división, grupos A-B

Profesor: Mansilla Muñoz York Elías

21 de octubre de 2026

v1.0.0 — Baseline aprobado

Taller de repaso y consolidación en C++: recursividad, búsqueda secuencial y punteros

Introducción

Esta actividad consolida tres pilares del trabajo de bajo nivel en C++: la recursividad, con su pila de llamadas y el caso base que evita el desbordamiento de pila; la búsqueda secuencial en arreglos de memoria contigua, optimizada con `break`; y los punteros, con pasaje por dirección y desreferenciación para intercambiar variables originales.

Se aplicó el método 5R (Registrar, Reducir, Responder, Reflexionar y Revisar) sobre una suite unificada con los tres retos. Integrante: Alexis Maier (trabajo individual). El control de cambios se documenta en `docs/CHANGELOG.md` y la tabla de revisiones figura en el desarrollo.

Desarrollo

Tabla de control de revisiones (DocVer)

Versión v1.0.0 — Baseline aprobado — 21 de octubre de 2026 — Autor: Alexis Maier — Primera versión completa del código, capturas, manuales e informe.

Método 5R aplicado

R1 Registrar: se compiló y ejecutó el código base con `g++ src/main.cpp -o src/repasogeneral.exe`. **R2 Reducir:** el núcleo de cada reto es el `if` del caso base (`n <= 0`), el `break` de la búsqueda y la variable `temporal` del intercambio. **R3 Responder:** se contestó el cuestionario de control. **R4 Reflexionar:** la lógica de búsqueda secuencial y los punteros se pueden aplicar a la lectura de lecturas de sensores almacenadas en buffers del prototipo. **R5 Revisar:** el código compila sin errores ni advertencias y quedó registrado en Git.

Ejecución de los tres retos

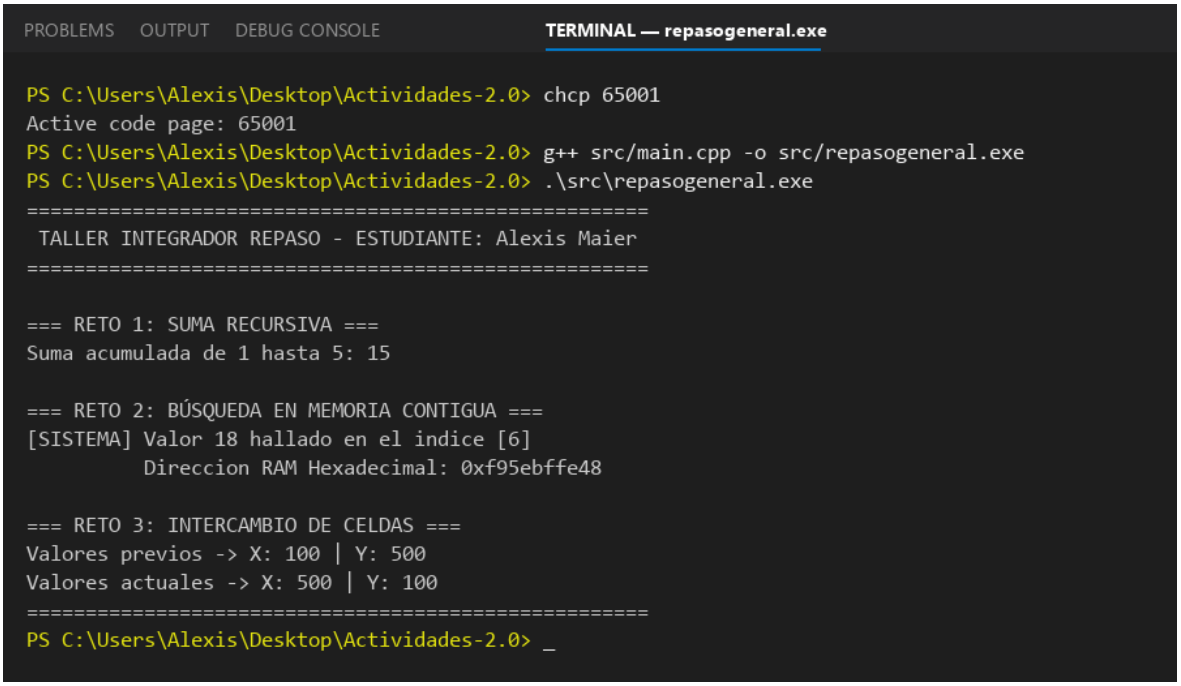


Figura 1. Compilación y ejecución de la suite con los tres retos.

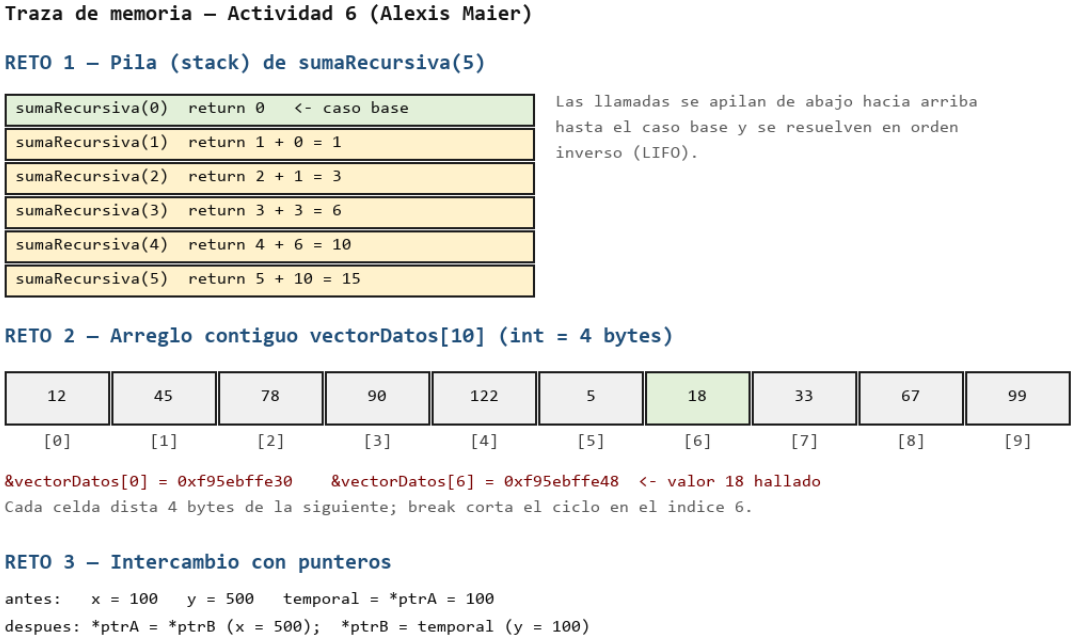


Figura 2. Traza de memoria: pila de llamadas recursivas, arreglo contiguo e intercambio.

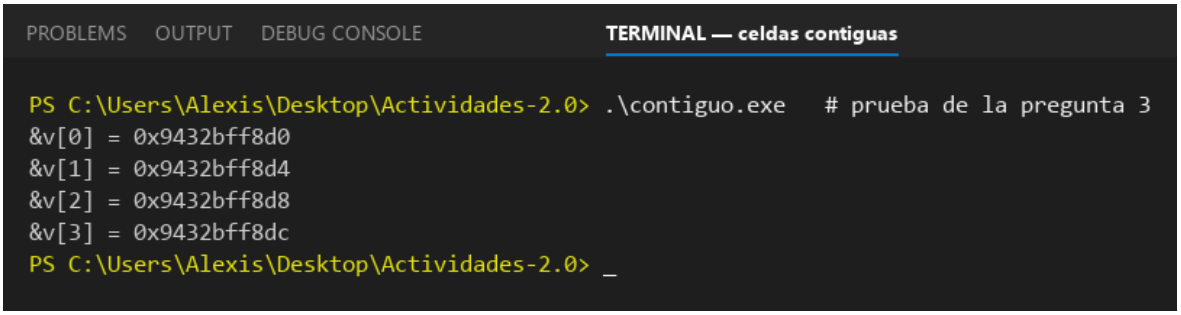


Figura 3. Direcciones de las celdas de un arreglo de int: cada una dista 4 bytes de la anterior.

Estrategia anti-caídas: GitHub + GitLab (Dual-Remote)

El proyecto se versiona en GitHub (`origin`). Para el respaldo en GitLab se debe crear un proyecto vacío con el mismo nombre y agregarlo como segundo destino de push con `git remote set-url --add --push origin <url>`, de modo que un único `git push` actualice ambos servidores. La verificación se hace con `git remote -v`, que debe mostrar una línea de fetch y dos de push. Este paso requiere la cuenta de GitLab del estudiante y figura como pendiente en el CHANGELOG.

Cuestionario técnico de control

1. Caso base: ¿Qué error crítico ocurre en la memoria RAM si una función recursiva carece de un caso base de corte?

Cada llamada recursiva reserva un nuevo registro de activación en la pila (stack) con sus variables locales y su dirección de retorno. Sin caso base, la función se sigue llamando indefinidamente, la pila se llena y el sistema operativo termina el programa por desbordamiento de pila (stack overflow, «segmentation fault» o código 0xC00000FD en Windows).

2. Eficiencia: ¿Por qué la instrucción break optimiza el rendimiento del microprocesador al realizar una búsqueda en arreglos?

Porque corta el ciclo en cuanto se encuentra el valor. Sin `break`, el procesador seguiría comparando las celdas restantes aunque el resultado ya esté determinado. En el mejor caso (valor en la primera celda) se hace 1 comparación en lugar de N; en promedio se evitan la mitad de las comparaciones, aunque la complejidad en el peor caso sigue siendo O(n).

3. Direccionamiento: ¿Cómo comprobar mediante el operador & que los elementos de un array ocupan celdas contiguas en la RAM?

Imprimiendo `&vector[0]`, `&vector[1]`, `&vector[2]`, etc., y verificando que cada dirección supera a la anterior en `sizeof(int)` (4 bytes). Se comprobó con un programa de prueba, cuya salida se muestra en la Figura 3.

4. Variables temporales: ¿Qué función cumple la variable temporal en el algoritmo de intercambio de valores con punteros?

Guarda una copia de `*ptrA` antes de sobrescribirlo. Sin ella, la instrucción `*ptrA = *ptrB;` destruiría el valor original de A y no podría asignarse a B (ambas celdas quedarían con el mismo valor).

5. Autoevaluación: ¿Cuál de los 3 retos te resultó más complejo de comprender a bajo nivel y de qué manera lograste resolverlo?

El más complejo fue la recursividad, porque cuesta visualizar que cada llamada queda «pendiente» en la pila hasta que se alcanza el caso base. Lo resolví dibujando la traza de la memoria (Figura 2): sumaRecursiva(5) espera a sumaRecursiva(4), y así hasta sumaRecursiva(0), que devuelve 0 y permite resolver las llamadas pendientes en orden inverso.

Conclusión

Los tres retos muestran cómo el programa se comporta a bajo nivel: la pila crece con cada llamada recursiva, la búsqueda secuencial recorre celdas consecutivas de la RAM y los punteros permiten modificar las variables originales. Documentar con una traza de memoria y registrar cada versión en el CHANGELOG hace reproducible el trabajo y prepara la integración con el Proyecto Integrador Anual.

Referencias

cplusplus.com. (s. f.). *Functions: Recursivity*. https://cplusplus.com/doc/tutorial/functions/

GeeksforGeeks. (s. f.). *Linear search*. https://www.geeksforgeeks.org/linear-search/

Git. (s. f.). *git-remote documentation*. https://git-scm.com/docs/git-remote

Mansilla Muñoz, Y. E. (2026). *Actividad 6: Taller práctico de repaso y consolidación en C++* [Guía de actividad]. E.E.S.T. N.º 1 «Eduardo Ader».